

Виявлення мережних атак

Кувавина Софія

Завдання: Налаштувати систему виявлення атак (SNORT або Suricata), протестувати її роботу на наявних в комплекті сигнатурах трафіку та додати нову сигнатуру, що описує особливості деяких пакетів відповідно індивідуального завдання. При цьому треба якомога точніше враховувати особливості протоколу та контексту передачі даних (іншими словами, ідентифкація протоколу по номерам портів не буде достатньою).

Налаштування мережі.

Я використала 2 віртуальні машини: Kali1 та Kali2. Машина Kali1 виконувала роль IDS/IPS а Kali2 хостом зовнішньої мережі.

Перевірка мережевих інтерфейсів та IP-адрес на Kali1:

```
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:5a:2d:7e brd ff:ff:ff:ff:ff:ff
    inet 192.168.114.138/24 brd 192.168.114.255 scope global dynamic noprefixroute eth0
        valid_lft 1138sec preferred_lft 1138sec
    inet6 fe80::1380:6f95:faa5:5e72/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:5a:2d:92 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.107/24 brd 192.168.0.255 scope global dynamic noprefixroute eth1
        valid_lft 6537sec preferred_lft 6537sec
    inet6 fe80::f668:f64c:4fb6:aeab/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Я перевірила мережеві інтерфейси на Kali1, щоб переконатися, що вузол IDS/IPS має два активні інтерфейси та може працювати між двома мережевими сегментами

Перевірка мережевого інтерфейсу та IP-адреси на Kali2:

```

(kali@kali2)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:f0:90:a6 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.106/24 brd 192.168.0.255 scope global dynamic noprefixroute eth0
        valid_lft 6470sec preferred_lft 6470sec
    inet6 fe80::1380:6f95:faa5:5e72/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

Запуск системи Snort на вузлі Kali1:

```

(kali@kali)-[~]
$ sudo snort -c /etc/snort/snort.lua -i eth1

o")~   Snort++ 3.11.1.0

Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
    dce_smb
    output
    port_scan
    ips
    references
    js_norm
    file_id
    smtp
    gtp_inspect
    dce_http_server
    dce_udp
    s7complus
    rpc_decode
    pop
    normalizer
    netflow
    imap
    dns
    back_orifice
    arp_spoof
    stream_file
    stream_user
    stream_udp
    stream_tcp
    stream_icmp
    stream_ip
    stream

```

Тут я запускаю Snort на Kali1 з конфігураційним файлом snort.lua на інтерфейсі eth1. Цим я перевіряю, що система коректно стартує і готова до аналізу мережевого трафіку.

На скріншоті видно статистику після запуску Snort. Рядок total rules loaded: 219 показує, що система успішно завантажила 219 стандартних правил ще до додавання свого власного правила:

```
rule counts
  total rules loaded: 219
  text rules: 219
  option chains: 219
  chain headers: 1

service rule counts
  file_id: 219
  total: 219

fast pattern groups
  to_server: 1
  to_client: 1

search engine (ac_bnfa)
  instances: 2
  patterns: 438
  pattern chars: 2602
  num states: 1832
  num match states: 392
  memory scale: KB
  total memory: 71.2812
  pattern memory: 19.6484
  match list memory: 28.4375
  transition memory: 22.9453
appid: MaxRss diff: 2972
appid: patterns loaded: 300

pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
+ [0] eth1
^C** caught int signal
= stopping
```

Отже, Snort на Kali1 запустився успішно і перейшов у режим обробки пакетів.

Індивідуальні варіанти завдань: 1.11 Виявлення пакетів протоколу GRE

Створення локального правила Snort для аналізу трафіку GRE-тунелю:

```
kali@kali: ~  
File Actions Edit View Help  
GNU nano 8.7 /etc/snort/rules/local.rules  
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $  
#  
# LOCAL RULES  
#  
# This file intentionally does not come with signatures. Put your local  
# additions here.  
alert icmp 10.10.10.2 any -> 10.10.10.1 any (msg:"LOCAL ICMP inside GRE tunnel detected"; sid:1000002; rev:1;)  
# tunnel add gre1 mode gre local 192.168.0.107 remote 192.168.0.106 ttl 255
```

Я створила локальне правило у файлі local.rules, і цьому правилі я налаштувала Snort так, щоб він реагував на ICMP-пакети, які передаються не напряму, а всередині GRE-тунелю. Тобто правило потрібне для перевірки, чи може система аналізувати інкапсульований трафік після проходження через GRE-заголовок. По суті, тут перевіряється не просто факт наявності GRE, а саме те, що всередині тунелю передається ICMP-трафік, наприклад ping.

Налаштування GRE-тунелю на вузлі Kali1:

```
(kali@kali)-[~]  
$ sudo modprobe ip_gre  
[sudo] password for kali:  
  
(kali@kali)-[~]  
$ sudo ip tunnel add gre1 mode gre local 192.168.0.107 remote 192.168.0.106 ttl 255  
  
(kali@kali)-[~]  
$ sudo ip addr add 10.10.10.1/30 dev gre1  
  
(kali@kali)-[~]  
$ sudo ip link set gre1 up  
  
(kali@kali)-[~]  
$ sudo ip link set gre1 up  
  
(kali@kali)-[~]  
$ ip a show gre1  
7: gre1@NONE: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1476 qdisc noqueue state UNKNOWN group def  
ault qlen 1000  
    link/gre 192.168.0.107 peer 192.168.0.106  
    inet 10.10.10.1/30 scope global gre1  
        valid_lft forever preferred_lft forever  
    inet6 fe80::c0a8:6b/64 scope link  
        valid_lft forever preferred_lft forever
```

Тут я завантажила модуль ip_gre, створила GRE-тунель на Kali1, призначила йому адресу 10.10.10.1/30 і перевірила, що інтерфейс gre1 з'явився та працює.

Налаштування GRE-тунелю на вузлі Kali2:


```

(kali@kali2)-[~]
$ sudo modprobe ip_gre

(kali@kali2)-[~]
$ sudo ip tunnel add gre1 mode gre local 192.168.0.106 remote 192.168.0.107 ttl 255

(kali@kali2)-[~]
$ sudo ip addr add 10.10.10.2/30 dev gre1

(kali@kali2)-[~]
$ sudo ip link set gre1 up

(kali@kali2)-[~]
$ ip a show gre1
6: gre1@NONE: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1476 qdisc noqueue state UNKNOWN group default qlen 1000
    link/gre 192.168.0.106 peer 192.168.0.107
    inet 10.10.10.2/30 scope global gre1
        valid_lft forever preferred_lft forever
    inet6 fe80::c0a8:6a/64 scope link
        valid_lft forever preferred_lft forever

```

На Kali2 я також налаштувала GRE-тунель, задала адресу 10.10.10.2/30 і перевірила, що інтерфейс gre1 створився коректно та готовий до передавання трафіку.

Запуск Snort після додавання користувацького правила:

```

finished rule args:

```

ips	policies	rule	stats
id	loaded	shared	enabled
0	220	0	220
file /etc/snort/snort.lua			

```

rule counts
total rules loaded: 220
text rules: 220
option chains: 220
chain headers: 2

```

port	rule counts	tcp	udp	icmp	ip
any	1	1	1	1	1
total	1	1	1	1	1

```

service rule counts
to-srv to-cli
file_id: 219 219
total: 219 219

```

fast pattern groups	to_server	to_client
to_server: 1	1	1
to_client: 1	1	1

```

search engine (ac_bnfa)
instances: 2
patterns: 438
pattern chars: 2602
num states: 1832
num match states: 392
memory scale: KB
total memory: 71.2812
pattern memory: 19.6484
match list memory: 28.4375
transition memory: 22.9453
appid: MaxRss diff: 3200
appid: patterns loaded: 300

```

Після додавання правила я повторно запустила Snort і перевірила, що нова сигнатура успішно підвантажилась. На скріншоті видно рядок `total rules loaded: 220`, тобто кількість правил збільшилась на одне.

Спрацювання користувацького правила Snort під час аналізу трафіку GRE-тунелю:

```
(kali@kali)-[~]
$ sudo snort -q -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r gre-test.pcap -A alert_talos

##### gre-test.pcap #####
[1:1000002:1] LOCAL ICMP inside GRE tunnel detected (alerts: 4)
#####
```

На скріншоті видно аналіз файлу `gre-test.pcap` у Snort і повідомлення `LOCAL ICMP inside GRE tunnel detected`. Також видно `alerts: 4`, тобто правило спрацювало чотири рази на трафіку, який передавався через GRE-тунель.

Перехоплення GRE-пакетів через `tcpdump`:

```
(kali@kali)-[~]
$ sudo tcpdump -ni eth1 proto gre
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
14:53:58.043517 IP 192.168.0.106 > 192.168.0.107: GREv0, length 88: IP 10.10.10.2 > 10.10.10.1: ICMP echo request, id 7, seq 1, length 64
14:53:58.043620 IP 192.168.0.107 > 192.168.0.106: GREv0, length 88: IP 10.10.10.1 > 10.10.10.2: ICMP echo reply, id 7, seq 1, length 64
14:53:59.058599 IP 192.168.0.106 > 192.168.0.107: GREv0, length 88: IP 10.10.10.2 > 10.10.10.1: ICMP echo request, id 7, seq 2, length 64
14:53:59.058740 IP 192.168.0.107 > 192.168.0.106: GREv0, length 88: IP 10.10.10.1 > 10.10.10.2
```

Тут я перехопила трафік на інтерфейсі `eth1` за допомогою `tcpdump`. Це дало змогу підтвердити, що в мережі дійсно присутні GRE-пакети, а всередині тунелю передається ICMP-трафік.

У цьому пункті я налаштувала Snort на Kali1, перевірила стандартні правила і додала власну сигнатуру. Після цього створила GRE-тунель між Kali1 і Kali2, згенерувала тестовий трафік і проаналізувала його. У результаті Snort розпізнав GRE-пакети, а моє правило успішно спрацювало.

2.11 Виявлення повідомлень HTTP GET

Створення локального правила Snort для виявлення повідомлень HTTP GET

```
kali@kali: ~
File Actions Edit View Help
GNU nano 8.7 /etc/snort/rules/local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# LOCAL RULES
# This file intentionally does not come with signatures. Put your local
# additions here.
alert icmp 10.10.10.2 any -> 10.10.10.1 any (msg:"LOCAL ICMP inside GRE tunnel detected"; sid:1000002; rev:1;)
alert tcp any any -> any 8000 (msg:"LOCAL HTTP GET detected"; content:"GET /"; sid:1000003; rev:4;)
```

Я додала до файлу local.rules користувацьке правило. Це правило призначене для виявлення HTTP-запитів типу GET у мережевому трафіку, воно спрацьовує тоді, коли клієнт звертається до сервера і просить віддати якийсь ресурс, наприклад сторінку або файл. Таке правило зручно використовувати для базового контролю веб-трафіку та щоб розуміти, які HTTP-запити проходять мережею

Запуск тестового HTTP-сервера на вузлі Kali1:

```
(kali@kali)-[~]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.0.106 - - [14/Mar/2026 15:46:03] "GET / HTTP/1.1" 200 -
192.168.0.106 - - [14/Mar/2026 15:47:19] "GET / HTTP/1.1" 200 -
```

Тут я запускаю тестовий HTTP-сервер на Kali1, щоб створити трафік для перевірки правила

Формування HTTP GET-запиту з вузла Kali2:

```
(kali@kali2)-[~]
$ curl http://192.168.0.107:8000/
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href=".android/">.android/</a></li>
<li><a href=".bash_logout">.bash_logout</a></li>
<li><a href=".bashrc">.bashrc</a></li>
<li><a href=".bashrc.original">.bashrc.original</a></li>
<li><a href=".cache/">.cache/</a></li>
<li><a href=".config/">.config/</a></li>
<li><a href=".dbus/">.dbus/</a></li>
<li><a href=".dmrc">.dmrc</a></li>
<li><a href=".face">.face</a></li>
<li><a href=".face.icon">.face.icon@</a></li>
<li><a href=".gnupg/">.gnupg/</a></li>
<li><a href=".gradle/">.gradle/</a></li>
<li><a href=".gvfs/">.gvfs/</a></li>
<li><a href=".ICEauthority">.ICEauthority</a></li>
<li><a href=".idapro/">.idapro/</a></li>
<li><a href=".java/">.java/</a></li>
<li><a href=".local/">.local/</a></li>
<li><a href=".mitmproxy/">.mitmproxy/</a></li>
<li><a href=".mozilla/">.mozilla/</a></li>
<li><a href=".msf4/">.msf4/</a></li>
<li><a href=".npm/">.npm/</a></li>
<li><a href=".profile">.profile</a></li>
<li><a href=".ssh/">.ssh/</a></li>
<li><a href=".sudo_as_admin_successful">.sudo_as_admin_successful</a></li>
<li><a href=".wget-hsts">.wget-hsts</a></li>
<li><a href=".Xauthority">.Xauthority</a></li>
<li><a href=".xsession-errors">.xsession-errors</a></li>
<li><a href=".xsession-errors.old">.xsession-errors.old</a></li>
<li><a href=".zprofile">.zprofile</a></li>
<li><a href=".zsh_history">.zsh_history</a></li>
<li><a href=".zshrc">.zshrc</a></li>
```

На цьому кроці я виконала HTTP GET-запит з Kali2 до HTTP-сервера на Kali1, щоб сформувати тестовий HTTP-трафік

Перехоплення HTTP-трафіку на інтерфейсі eth1 у файл http-get.pcap:


```
(kali@kali)-[~]
$ sudo tcpdump -ni eth1 port 8000 -w http-get.pcap
[sudo] password for kali:
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C14 packets captured
14 packets received by filter
0 packets dropped by kernel
```

Після цього я перехопила HTTP-трафік на інтерфейсі eth1 за допомогою tcpdump і зберегла його у файл http-get.pcap для подальшого аналізу в Snort.

Спрацювання користувацького правила Snort на повідомлення HTTP GET:

```
(kali@kali)-[~]
$ sudo snort -q -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r http-get.pcap -A alert_talos
##### http-get.pcap #####
[1:1000003:4] LOCAL HTTP GET detected (alerts: 1)
#####
```

Тут показано аналіз файла http-get.pcap у Snort. На скріншоті видно, що створене правило спрацювало саме на HTTP GET-запит.

У цьому пункті я створила правило Snort для виявлення HTTP GET-повідомлень, запустила тестовий HTTP-сервер і сформуvalа запит з Kali2. Далі перехопила трафік у pcap-файл та перевірила його в Snort. У результаті правило спрацювало коректно, тобто HTTP GET-повідомлення успішно виявляються.

3.11 Виявлення повідомлень протоколу PPTP

Додавання локального правила для виявлення повідомлень PPTP:

```
#alert tcp 192.168.0.107 8000 → 192.168.0.106 any (msg: "
alert tcp 192.168.0.106 any → 192.168.0.107 1723 (
  msg:"LOCAL PPTP control message detected";
  flow:to_server;
  raw_data;
  content:"|00 9C 00 01 1A 2B 3C 4D 00 01 00 00|";
  sid:1000004;
  rev:3;
)
```

У цьому правилі я налаштувала виявлення повідомлень протоколу PPTP, а саме його керуючої частини. Такі пакети зазвичай передаються через TCP-порт 1723 і містять характерні службові поля. Правило шукає саме ознаки PPTP control message в пакеті, щоб відрізнити його від звичайного TCP-трафіку.

Формування тестового pcap-файлу для перевірки RPTP-повідомлення:

```
GNU nano 8.7 3-11.py
from scapy.all import Ether, IP, TCP, Raw, wrpcap

src_ip = "192.168.0.106"
dst_ip = "192.168.0.107"

pptp_payload = (
    b"\x00\x9c"          # Length
    b"\x00\x01"          # PPTP Message Type = Control Message
    b"\x1A\x2B\x3C\x4D"  # Magic Cookie
    b"\x00\x01"          # Control Message Type = Start-Control-Connection-Request
    b"\x00\x00"          # Reserved
    + b"A" * 144
)

pkt = (
    Ether() /
    IP(src=src_ip, dst=dst_ip) /
    TCP(sport=40000, dport=1723, flags="PA", seq=1, ack=1) /
    Raw(load=pptp_payload)
)

wrpcap("3-11.pcap", [pkt])
print("Created 3-11.pcap")
```

Для перевірки правила я підготувала скрипт 3-11.py. Цей скрипт формує тестовий TCP-пакет на порт 1723, у payload якого вручну задані характерні поля RPTP control message.

Після запуску скрипта 3-11.py було створено файл 3-11.pcap, який містить тестовий пакет для подальшого аналізу в системі Snort:

```
(kali@kali)-[~]
$ python3 3-11.py
WARNING: getmacbyip failed on [Errno 1] Operation not permitted
WARNING: MAC address to reach destination not found. Using broadcast.
Created 3-11.pcap

(kali@kali)-[~]
$ ls -l 3-11.pcap
-rw-rw-r-- 1 kali kali 250 Mar 15 13:54 3-11.pcap
```

Перевірка вмісту сформованого pcap-файлу:

```

(kali@kali)-[~]
$ tcpdump -nn -X -r 3-11.pcap
reading from file 3-11.pcap, link-type EN10MB (Ethernet), snapshot length 65535
13:54:15.491913 IP 192.168.0.106.40000 > 192.168.0.107.1723: Flags [P.], seq 1:157, ack 1, win 8192, length 156: ptp CTRL_MSGTYPE=SCCRQ PROTO_VER(65.65) [E
RROR: reserved=16705 must be zero] FRAME_CAP(A) BEARER_CAP(A) MAX_CHAN(16705) FIRM_REV(16705) HOSTNAME(AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAA) VENDOR(AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA)
0x0000: 4500 00c4 0001 0000 4006 f80d c0a8 006a E.....@.....j
0x0010: c0a8 006b 9c40 06bb 0000 0001 0000 0001 ...k.@.....
0x0020: 5018 2000 b89c 0000 009c 0001 1a2b 3c4d P.....+<M
0x0030: 0001 0000 4141 4141 4141 4141 4141 4141 ....AAAAAAAAAAAA
0x0040: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAAAA
0x0050: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAAAA
0x0060: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAAAA
0x0070: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAAAA
0x0080: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAAAA
0x0090: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAAAA
0x00a0: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAAAA
0x00b0: 4141 4141 4141 4141 4141 4141 4141 4141 AAAAAAAAAAAAAAAAAA
0x00c0: 4141 4141 AAAA

```

Під час перегляду вмісту 3-11.pcap було видно, що пакет спрямований на TCP-порт 1723 і містить характерні ознаки PPTP control message. Зокрема, tcpdump розпізнав його як ptp CTRL_MSGTYPE=SCCRQ, а в hex-представленні видно послідовність 1A 2B 3C 4D, яка відповідає Magic Cookie протоколу PPTP.

Перевірка спрацювання правила Snort на файлі 3-11.pcap:

```

(kali@kali)-[~]
$ sudo snort -q -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r 3-11.pcap -A alert_fast
03/15-13:54:15.491913 [**] [1:1000004:3] "LOCAL PPTP control message detected" [**] [Priority: 0] {TCP} 192.168.0.106:40000 → 192.168.0.107:1723

```

Під час перегляду вмісту 3-11.pcap видно, що пакет спрямований на TCP-порт 1723 і має характерні ознаки PPTP control message. Зокрема, tcpdump розпізнає його як ptp CTRL_MSGTYPE=SCCRQ, а в hex-представленні видно послідовність 1A 2B 3C 4D - це Magic Cookie протоколу PPTP.

У пункті 3.11 я реалізувала виявлення PPTP-повідомлень за допомогою локального правила Snort. Для перевірки створила тестовий pcap-файл із характерними полями PPTP control message, переглянула його в tcpdump і переконалася, що потрібні ознаки справді присутні.

4.11 Виявлення пакетів GRE тунелів

Додавання локального правила Snort для аналізу трафіку GRE-тунелю:

```
kali@kali: ~
File Actions Edit View Help
GNU nano 8.7 /etc/snort/rules/local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
#
# LOCAL RULES
#
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert icmp 10.10.10.2 any -> 10.10.10.1 any (msg:"LOCAL ICMP inside GRE tunnel detected"; sid:1000002; rev:1;)
#alert tcp any any -> any 8000 (msg:"LOCAL HTTP GET detected"; content:"GET /"; sid:1000003; rev:4;)
#alert tcp any any -> any 1723 (msg:"LOCAL PPTP detected"; sid:1000004; rev:1;)
alert ip 192.168.0.106 any -> 192.168.0.107 any (msg:"LOCAL GRE tunnel packet detected"; ip_proto:47; sid:1000005; rev:1;)
```

У цьому правилі я налаштувала перевірку IP-пакетів за номером протоколу в заголовку. Для GRE використовується значення 47, тому саме за цією ознакою Snort може визначити, що пакет належить до GRE-трафіку. Таке правило дозволяє виділити пакети тунелювання серед іншого мережевого трафіку.

Перевірка налаштованого GRE-тунелю на вузлі Kali1:

```
(kali@kali)-[~]
$ ip a show gre1
7: gre1@NONE: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1476 qdisc noqueue state UNKNOWN group default qlen 1000
    link/gre 192.168.0.107 peer 192.168.0.106
    inet 10.10.10.1/30 scope global gre1
        valid_lft forever preferred_lft forever
    inet6 fe80::c0a8:6b/64 scope link
        valid_lft forever preferred_lft forever
```

Тут я перевірила стан інтерфейсу gre1 на Kali1, щоб переконатися, що GRE-тунель налаштований правильно і готовий до передавання трафіку.

Перехоплення пакетів GRE-тунелю на інтерфейсі eth1:

```
(kali@kali)-[~]
$ sudo tcpdump -ni eth1 proto gre -w gre-tunnel.pcap
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C8 packets captured
8 packets received by filter
0 packets dropped by kernel
```

На цьому етапі я перехопила GRE-пакети на інтерфейсі eth1 за допомогою tcpdump і зберегла їх у файл gre-tunnel.pcap.

Передача тестового трафіку через GRE-тунель;


```
(kali㉿kali2)-[~]  
$ ping -c 4 10.10.10.1  
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.  
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=9.93 ms  
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=1.92 ms  
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=1.09 ms  
64 bytes from 10.10.10.1: icmp_seq=4 ttl=64 time=1.41 ms  
  
— 10.10.10.1 ping statistics —  
4 packets transmitted, 4 received, 0% packet loss, time 3004ms  
rtt min/avg/max/mdev = 1.089/3.585/9.927/3.673 ms
```

Далі я згенерувала тестовий трафік через GRE-тунель командою ping, щоб перевірити передавання пакетів між Kali2 і Kali1 через тунель.

Аналіз файла gre-tunnel.pcap у Snort із розпізнаванням GRE-пакетів:

```
[0] gre-tunnel.pcap

Packet Statistics
-----
daq
pcaps: 1
received: 8
analyzed: 8
allow: 8
rx_bytes: 976

codec
total: 8 (100.000%)
eth: 8 (100.000%)
gre: 8 (100.000%)
icmp4: 8 (100.000%)
ipv4: 16 (200.000%)

Module Statistics
-----
appid
packets: 8
processed_packets: 8
total_sessions: 1

binder
new_flows: 1
inspects: 1

detection
analyzed: 8
hard_evals: 8
alerts: 4
total_alerts: 4
logged: 4
```

На скріншоті видно результати аналізу файла gre-tunnel.pcap у Snort. У блоці codec зафіксовано gre: 8, тобто Snort розпізнав пакети GRE-тунелю.

У цьому пункті я додала правило Snort для аналізу пакетів GRE-тунелю, перевірила наявність налаштованого тунелю між Kali1 і Kali2, згенерувала тестовий трафік і перехопила його через tcpdump. Далі я проаналізувала файл gre-tunnel.pcap у Snort.

Окреме повідомлення користувацького правила не було виведене, оскільки в документації Snort зазначено, що при обробці GRE система переходить до корисного навантаження після GRE-заголовка і логуванню підлягає саме інкапсульований трафік, а не зовнішній GRE-заголовок. Також документація Snort 3 вказує, що для таких правил використовується заголовок alert ip ... з перевіркою значення поля IP Protocol через ip_proto, а не окремий GRE-протокол у заголовку правила. але в статистиці codec видно gre: 8, тобто Snort коректно розпізнав GRE-трафік.

5.11 Виявлення пакетів протоколу GRE

Створення скрипта для формування pcap-файлу з пакетами протоколу GRE:

```
File Actions Edit View Help
GNU nano 8.7 5-11.py
from scapy.all import Ether, IP, GRE, ICMP, wrpcap

pkt = (
    Ether() /
    IP(src="192.168.0.106", dst="192.168.0.107") /
    GRE() /
    IP(src="10.10.10.2", dst="10.10.10.1") /
    GRE() /
    IP(src="172.16.1.2", dst="172.16.1.1") /
    ICMP()
)

wrpcap("5-11.pcap", [pkt])
print("5-11.pcap created")
```

Я створила Python-скрипт 5-11.py для формування pcap-файлу з тестовим пакетом протоколу GRE.

Після запуску скрипта було сформовано файл 5-11.pcap:

```
(kali㉿kali)-[~]
$ python3 5-11.py
WARNING: getmacbyip failed on [Errno 1] Operation not permitted
WARNING: MAC address to reach destination not found. Using broadcast.
5-11.pcap created
```

Аналіз файла 5-11.pcap у Snort:

```

(kali㉿kali)-[~]
└─$ sudo snort -c /etc/snort/snort.lua -r 5-11.pcap -A alert_fast

o")~   Snort++ 3.11.1.0

Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
  output
  search_engine
  file_id
  ips
  classifications
  references
  binder
  wizard
  appid
  js_norm
  file_policy
  http2_inspect
  http_inspect
  ftp_data
  ftp_client
  ftp_server
  smtp
  port_scan
  gtp_inspect
  dce_http_server
  dce_http_proxy
  dce_udp
  dce_tcp
  dce_smb
  s7commplus
  opcua
  modbus
  mms
  iec104
  telnet
  ssh
  rpc_decode
  pop
  normalizer

```

Результати аналізу файла 5-11.pcap у Snort із розпізнаванням пакетів протоколу GRE:


```
rx_bytes: 90
-----
codec
total: 1      (100.000%)
eth: 1        (100.000%)
gre: 2        (200.000%)
icmp4: 1      (100.000%)
ipv4: 3       (300.000%)
-----
Module Statistics
-----
appid
packets: 1
```

На скріншоті видно результати аналізу файла 5-11.pcap у Snort. У блоці codec зафіксовано gre: 2, тобто система розпізнала два GRE-заголовки у сформованому пакеті.

У цьому пункті я створила Python-скрипт для формування тестового pcap-файлу з пакетами GRE, згенерувала файл 5-11.pcap і проаналізувала його в Snort. У статистиці декодування з'явилося значення gre: 2, тому можна зробити висновок, що Snort коректно розпізнає пакети протоколу GRE.

6.11 Виявити пакети HTTP, в яких передаються файли (зберегти дані пакетів, якщо у використовуваній версії програмного забезпечення буде така можливість)

Створення тестового файла для передавання через HTTP:

```
(kali@kali)-[~]
$ echo "LAB6FILEMARKER_611" > secret_file.txt
cat secret_file.txt
ls -l secret_file.txt
LAB6FILEMARKER_611
-rw-rw-r-- 1 kali kali 19 Mar 15 12:55 secret_file.txt
```

Я створила тестовий файл secret_file.txt, який далі передавався через HTTP для перевірки виявлення передавання файлів у Snort.

Запуск тестового HTTP-сервера на вузлі Kali1:

```
(kali@kali)-[~]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Завантаження файла secret_file.txt через HTTP з вузла Kali2:

```
(kali@kali2)-[~]
$ curl -O http://192.168.0.107:8000/secret_file.txt
ls -l secret_file.txt
cat secret_file.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left   Speed
100    19100    100    19  0      0    362      0      0     0
-rw-rw-r-- 1 kali kali 19 Mar 15 12:56 secret_file.txt
LAB6FILEMARKER_611
```

На цьому кроці я завантажила файл secret_file.txt з Kali1 на Kali2 через HTTP, щоб сформувати тестовий трафік передавання файлу.

Перехоплення HTTP-трафіку у файл http-file.pcap:

```
(kali@kali)-[~]
$ sudo tcpdump -ni eth1 port 8000 -w http-file.pcap
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C10 packets captured
10 packets received by filter
0 packets dropped by kernel
```

Після цього я перехопила HTTP-трафік на інтерфейсі eth1 і зберегла його у файл http-file.pcap для подальшого аналізу.

Перевірка вмісту файлу http-file.pcap:

```
(kali@kali)-[~]
$ sudo tcpdump -A -r http-file.pcap | head -n 120
reading from file http-file.pcap, link-type EN10MB (Ethernet), snapshot length 262144
12:56:08.496643 IP 192.168.0.106.54670 > 192.168.0.107.8000: Flags [S], seq 1956706061, win 64240, options [mss 1460,sackOK,TS val 4286476722 ecr 0,nop,wscale 7], length 0
E..<.%@.@...q...j...k...@t.....:.....
..q.....
12:56:08.497960 IP 192.168.0.107.8000 > 192.168.0.106.54670: Flags [S.], seq 1795937480, ack 1956706062, win 65160, options [mss 1460,sackOK,TS val 1986120104 ecr 4286476722,nop,wscale 7], length 0
E..<..@.@.....k...j..@..k...t.....T.....
va...~q.....
12:56:08.498707 IP 192.168.0.106.54670 > 192.168.0.107.8000: Flags [.], ack 1, win 502, options [nop,nop,TS val 4286476724 ecr 1986120104], length 0
E..4.6@.@..x...j...k...@t...k.....
..q.va..
12:56:08.499179 IP 192.168.0.106.54670 > 192.168.0.107.8000: Flags [P.], seq 1:98, ack 1, win 502, options [nop,nop,TS val 4286476725 ecr 1986120104], length 97
E....'@.@.....j...k...@t...k.....
..q.va..GET /secret_file.txt HTTP/1.1
Host: 192.168.0.107:8000
User-Agent: curl/8.18.0
Accept: */*

12:56:08.499268 IP 192.168.0.107.8000 > 192.168.0.106.54670: Flags [.], ack 98, win 509, options [nop,nop,TS val 1986120105 ecr 4286476725], length 0
E..49d@.@.:...k...j..@..k...t..o.....L.....
va...~q.
12:56:08.510015 IP 192.168.0.107.8000 > 192.168.0.106.54670: Flags [P.], seq 1:187, ack 98, win 509, options [nop,nop,TS val 1986120116 ecr 4286476725], length 186
E...9e@.@.~....k...j..@..k...t..o.....
va...~q.HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.13.7
```

На скріншоті видно вміст перехопленого HTTP-трафіку: HTTP GET-запит до файла secret_file.txt, відповідь сервера та маркер LAB6FILEMARKER_611 у переданому файлі. Тобто на цьому етапі підтверджується сам факт передавання файла через HTTP

Скрипт для формування тестового pcap-файлу:

```
GNU nano 8.7
from scapy.all import Ether, IP, TCP, wrpcap

payload = (
    b"HTTP/1.1 200 OK\r\n"
    b"Content-Type: text/plain\r\n"
    b"Content-Length: 19\r\n"
    b"\r\n"
    b"LAB6FILEMARKER_611"
)

pkt = (
    Ether() /
    IP(src="192.168.0.107", dst="192.168.0.106") /
    TCP(sport=8000, dport=55555, flags="PA", seq=1, ack=1) /
    payload
)

wrpcap("6-11.pcap", [pkt])
print("6-11.pcap created")
```

Я створила Python-скрипт 6-11.py для формування контрольного pcap-файлу з HTTP-відповіддю, у якій передається тестовий файл.

Далі я запустила скрипт і сформувала файл 6-11.pcap:

```
(kali@kali)-[~]
$ python3 6-11.py
ls -l 6-11.pcap
WARNING: getmacbyip failed on [Errno 1] Operation not permitted
WARNING: MAC address to reach destination not found. Using broadcast.
6-11.pcap created
-rw-rw-r-- 1 kali kali 177 Mar 15 13:17 6-11.pcap
```

Додавання локального правила для виявлення HTTP-передавання файла:

```

GNU nano 8.7 /etc/snort/rules/local.rules *
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
#
# LOCAL RULES
#
# This file intentionally does not come with signatures. Put your local
# additions here.
#alert icmp 10.10.10.2 any → 10.10.10.1 any (msg:"LOCAL ICMP inside GRE tunnel detected"; sid:1000002; rev:1;)
#alert tcp any any → any 8000 (msg:"LOCAL HTTP GET detected"; content:"GET /"; sid:1000003; rev:4;)
#alert tcp any any → any 1723 (msg:"LOCAL PPTP detected"; sid:1000004; rev:1;)
#alert ip 192.168.0.106 any → 192.168.0.107 any (msg:"LOCAL GRE tunnel packet detected"; ip_proto:47; sid:1000005; rev:1;)
#alert ip 192.168.0.106 any → 192.168.0.107 any (msg:"LOCAL GRE protocol packet detected"; ip_proto:47; sid:1000006; rev:1;)
#alert tcp 192.168.0.107 8000 → 192.168.0.106 any (msg:"LOCAL HTTP file transfer detected"; raw_data; content:"LAB6FILEMARKER_611"; sid:1000007; rev:6;)

```

У цьому правилі я налаштувала Snort так, щоб він реагував на HTTP-трафік, у якому передається файл. Для цього в правилі перевіряється характерний вміст пакета — або назва файла, або спеціальний маркер у тілі HTTP-відповіді. Тобто правило орієнтується не просто на факт HTTP-з'єднання, а на вміст переданих даних.

Спрацювання користувацького правила Snort на HTTP-передавання файла:

```

(kali@kali)-[~]
$ sudo snort -q -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r 6-11.pcap -A alert_talos
##### 6-11.pcap #####
      [1:1000007:6] LOCAL HTTP file transfer detected (alerts: 1)
#####

```

Тут показано аналіз файла 6-11.pcap у Snort. На скріншоті видно, що створене правило спрацює на HTTP-пакет, у якому передається файл.

У цьому пункті я створила тестовий файл і організувала його передавання через HTTP між Kali1 і Kali2. Потім перехопила трафік у файл http-file.pcap і перевірила його вміст, щоб підтвердити факт передавання файла. Для контрольної перевірки також сформуvala окремий файл 6-11.pcap з HTTP-відповіддю, яка містить файл. Після додавання локального правила Snort проаналізувала цей трафік, і правило успішно спрацювало.

Висновок:

У цій лабораторній роботі я на практиці розібралася, як працює Snort і як за допомогою правил можна виявляти різний мережевий трафік. Я перевірила виявлення ICMP у GRE-тунелі, HTTP GET-запитів, повідомлень PPTP, GRE-пакетів і передавання файла через HTTP. Під час роботи я додавала локальні правила, запускала аналіз pcap-файлів і дивилась, чи спрацює Snort на потрібні пакети. У результаті я побачила, що система справді дозволяє знаходити задані типи трафіку та контролювати, що саме відбувається в мережі. Отже, робота допомогла мені краще зрозуміти принцип сигнатурного аналізу трафіку і закріпити навички роботи зі Snort.